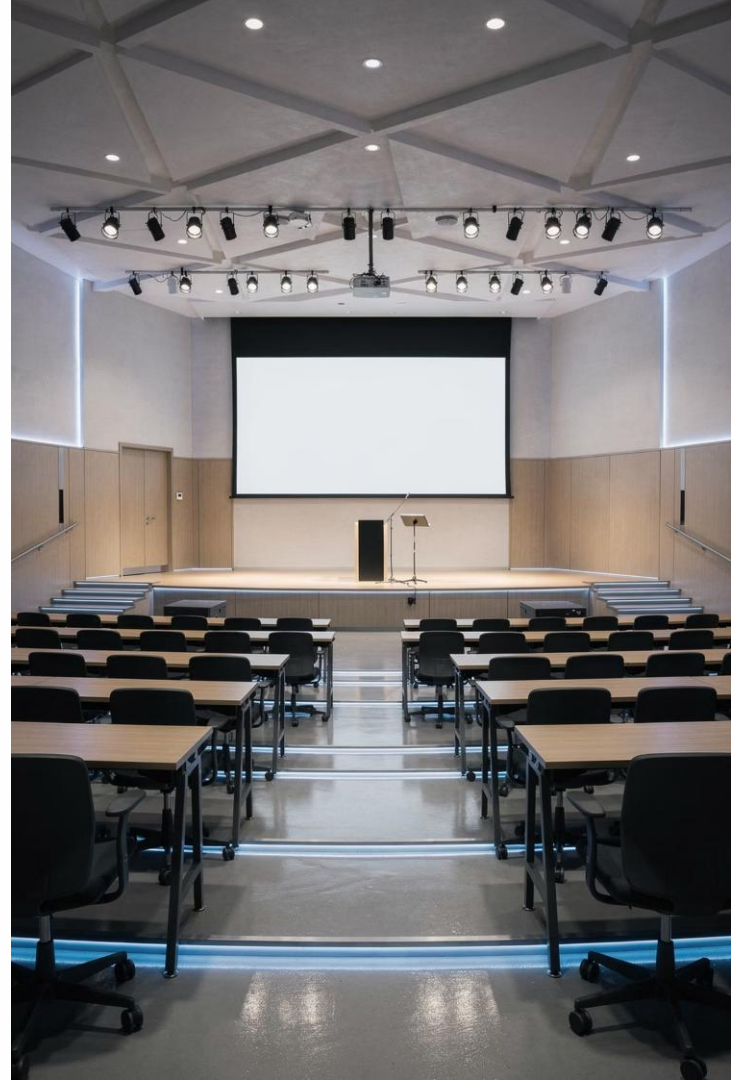


Patrón Arquitectónico MVC y Modelado UML Aplicado

Introducción a la Programación II ·



¿Qué aprenderás hoy?

Al finalizar esta sesión serás capaz de comprender, analizar y aplicar el patrón MVC junto con las herramientas de modelado UML en el contexto del desarrollo de software profesional.

1

Comprender MVC

Identificar los conceptos, ventajas y problemas que resuelve el patrón arquitectónico Modelo-Vista-Controlador.

2

Analizar cada capa

Diferenciar las responsabilidades del Modelo, la Vista y el Controlador con ejemplos reales de software.

3

Modelar con UML

Crear diagramas de clases, secuencias y componentes aplicados directamente a la arquitectura MVC.

4

Aplicar en casos reales

Resolver casos prácticos de sistemas CRUD, tiendas en línea y gestión académica con MVC.

¿Por Qué Utilizar Patrones Arquitectónicos?

En el desarrollo de software real, los proyectos crecen en complejidad rápidamente. Sin una arquitectura bien definida, el código se vuelve difícil de mantener, probar y escalar. Los patrones arquitectónicos ofrecen soluciones probadas a estos problemas recurrentes.

Sin patrón arquitectónico

- Código mezclado: UI, lógica y datos en un mismo archivo
- Difícil de mantener y depurar
- Imposible de escalar en equipo
- Las pruebas unitarias son inviables

Con patrón arquitectónico

- Responsabilidades claramente separadas por capas
- Código reutilizable y mantenible
- Equipos pueden trabajar en paralelo
- Pruebas unitarias e integración facilitadas

¿Qué es el Patrón MVC?

El **Modelo-Vista-Controlador (MVC)** es un patrón arquitectónico que divide una aplicación en tres componentes principales, separando la lógica de negocio, la presentación y el control del flujo. Fue introducido por Trygve Reenskaug en 1979 y hoy es el estándar en frameworks como Laravel (PHP), Spring MVC (Java), Ruby on Rails y Django (Python).

Modelo (Model)

Gestiona los datos, la lógica de negocio y la comunicación con la base de datos. Es el núcleo de la aplicación.

Vista (View)

Presenta la información al usuario mediante interfaces gráficas, formularios web o plantillas HTML.

Controlador (Controller)

Recibe las solicitudes del usuario, coordina el Modelo y selecciona la Vista apropiada para responder.

  **Nota docente:** Usa la analogía del restaurante: el cliente (Vista), el mesero (Controlador) y la cocina (Modelo). — Tiempo: 8 min.

El Problema: Código Espagueti Sin MVC

Observa el contraste entre un enfoque sin arquitectura y uno con MVC. En el código sin MVC, toda la lógica, presentación y acceso a datos conviven en un único archivo, generando acoplamiento total y deuda técnica severa.

✗ Sin MVC — index.php típico

```
// Consulta SQL directa en la vista
$result = mysqli_query($conn,
    "SELECT * FROM usuarios");

// HTML mezclado con PHP
echo "<table>";
while($row = mysqli_fetch_array($result)){
    echo "<tr><td>".$row['nombre']."</td></tr>";
}
echo "</table>";

// Lógica de negocio en el mismo archivo
if($row['edad'] < 18) {
    echo "Acceso denegado";
}
```

✓ Con MVC — Separación clara

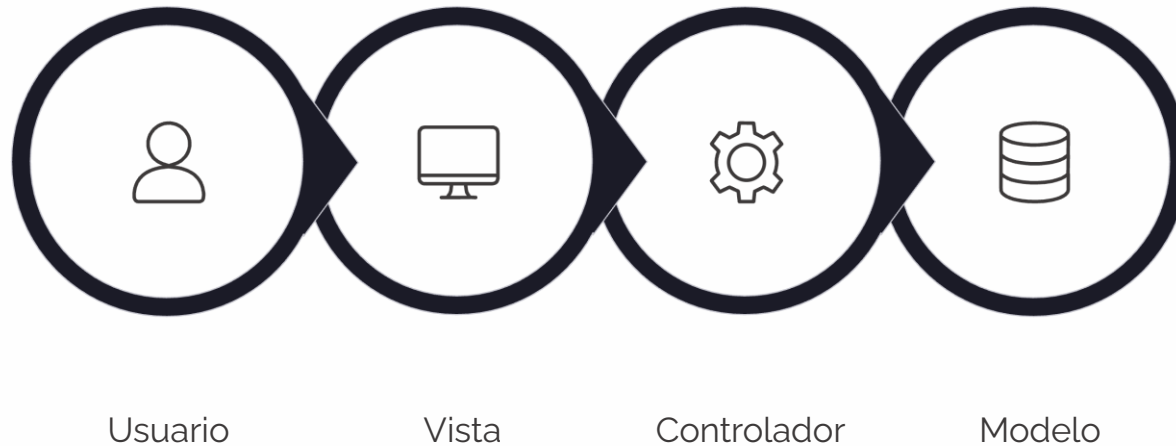
```
// Model: UsuarioModel.php
class UsuarioModel {
    public function getAll(): array {
        return $this->db->query(
            "SELECT * FROM usuarios"
        )->fetchAll();
    }
}

// Controller: UsuarioController.php
public function listar() {
    $usuarios = $this->model->getAll();
    $this->view->render('lista', $usuarios);
}

// View: lista.html.twig
{% for u in usuarios %}
    <tr><td>{{ u.nombre }}</td></tr>
{% endfor %}
```

Arquitectura General del Patrón MVC

El flujo de comunicación en MVC sigue una dirección definida. El usuario interactúa con la **Vista**, que delega la acción al **Controlador**. Éste procesa la solicitud, invoca al **Modelo** para obtener o modificar datos, y finalmente actualiza la Vista con la respuesta.



Este flujo unidireccional garantiza que cada capa tenga una responsabilidad única, lo que se conoce como el principio de **Separación de Responsabilidades (SoC)**. Ninguna capa debe conocer los detalles internos de otra.

El Modelo (Model): Corazón de la Aplicación

El Modelo es la capa responsable de toda la **lógica de negocio y el acceso a datos**. Representa las entidades del dominio, aplica las reglas de negocio y se comunica directamente con la base de datos mediante consultas SQL o un ORM (Object-Relational Mapper).

Responsabilidades del Modelo

- Representar las entidades del negocio (Usuario, Producto, Pedido)
- Ejecutar consultas CRUD sobre la base de datos
- Aplicar validaciones y reglas de negocio
- Notificar cambios de estado a la Vista

Ejemplo: Modelo de Estudiante (Java)

```
public class EstudianteModel {  
    private int id;  
    private String nombre;  
    private String correo;  
  
    public List<Estudiante> listarTodos() {  
        String sql = "SELECT * FROM estudiantes";  
        return db.executeQuery(sql);  
    }  
  
    public boolean registrar(Estudiante e) {  
        String sql = "INSERT INTO estudiantes "  
            + "VALUES(?, ?, ?)";  
        return db.execute(sql, e.getNombre(),  
            e.getCorreo());  
    }  
}
```

La Vista (View): Interfaz con el Usuario

La Vista es la capa de **presentación**. Su única responsabilidad es mostrar los datos que recibe del Controlador al usuario final. En aplicaciones web, la Vista se implementa mediante plantillas HTML, motores de plantillas (Twig, Blade, Thymeleaf) o frameworks de frontend (React, Angular, Vue).

Responsabilidades de la Vista

- Renderizar datos proporcionados por el Controlador
- Presentar formularios y capturar entrada del usuario
- No contener lógica de negocio ni acceso a datos
- Ser intercambiable (web, móvil, API REST)



✓ Una buena Vista es "tonta": solo muestra lo que recibe, sin calcular nada.

Ejemplo: Vista HTML con Blade (Laravel)

```
{{-- resources/views/estudiantes/lista.blade.php --}}
<table class="table">
  <thead>
    <tr>
      <th>ID</th>
      <th>Nombre</th>
      <th>Correo</th>
    </tr>
  </thead>
  <tbody>
    @foreach($estudiantes as $est)
      <tr>
        <td>{{ $est->id }}</td>
        <td>{{ $est->nombre }}</td>
        <td>{{ $est->correo }}</td>
      </tr>
    @endforeach
  </tbody>
</table>
```

El Controlador (Controller): Director de Orquesta

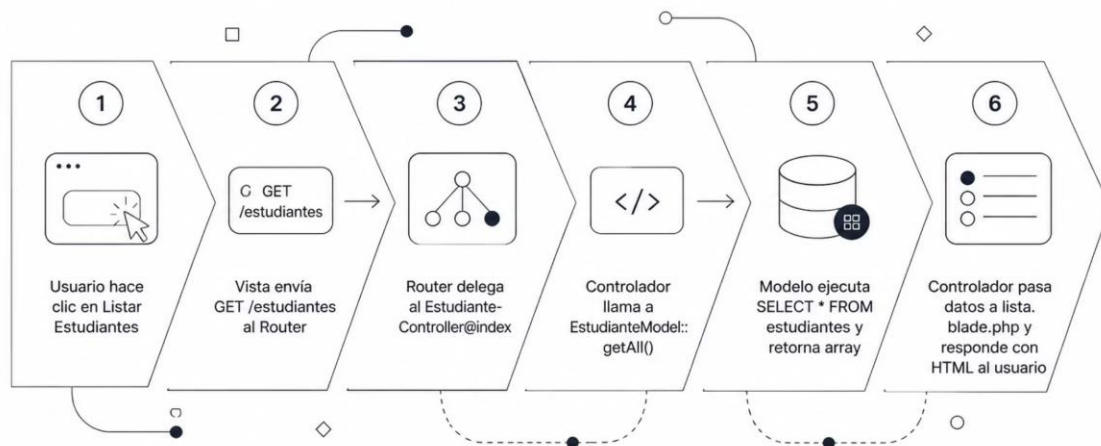
El Controlador actúa como **intermediario entre la Vista y el Modelo**. Recibe las solicitudes HTTP del usuario, determina qué lógica de negocio debe ejecutarse, invoca al Modelo correspondiente y selecciona la Vista adecuada para presentar la respuesta. Es el director de orquesta de la aplicación.

01	02
Recibe la solicitud	Valida y procesa
Captura la petición HTTP (GET, POST, PUT, DELETE) con todos sus parámetros y datos de sesión.	Valida los datos de entrada, aplica lógica de flujo y determina qué acción del Modelo invocar.
03	04
Consulta el Modelo	Selecciona la Vista
Llama al Modelo para obtener, guardar o modificar datos según la acción solicitada por el usuario.	Pasa los datos obtenidos del Modelo a la Vista correspondiente y la renderiza como respuesta al usuario.

```
// EstudianteController.php
public function registrar(Request $request) {
    $data = $request->validate(['nombre'=>'required', 'correo'=>'email']);
    $this->model->guardar($data);           // Invoca Modelo
    return redirect()->route('lista')      // Selecciona Vista
        ->with('exito', 'Estudiante registrado');
}
```

Flujo Completo MVC: De la Solicitud a la Respuesta

A continuación se muestra el recorrido completo de una solicitud en un sistema MVC real, desde que el usuario hace clic en un botón hasta que recibe la respuesta visual en su navegador.



Regla de Oro

La Vista nunca habla con el Modelo directamente. Todo pasa por el Controlador.

Ventaja clave

Puedes cambiar la Vista (HTML → API JSON) sin tocar el Modelo ni el Controlador.

Testabilidad

Puedes probar el Modelo y el Controlador de forma independiente con pruebas unitarias.

Actividad Rápida: ¿A qué capa pertenece?

Clasifica cada fragmento de funcionalidad en su capa MVC correspondiente. Trabaja individualmente durante 3 minutos y luego comparte tu respuesta en el chat.

Funcionalidad A

"Calcular el descuento del 15% sobre el precio total del carrito cuando el cliente tiene membresía premium."

¿Modelo, Vista o Controlador?

Funcionalidad B

"Mostrar el mensaje de error 'Usuario no encontrado' en color rojo debajo del campo de correo electrónico."

¿Modelo, Vista o Controlador?

Funcionalidad C

"Recibir el formulario de registro, validar que el email no exista y redirigir al dashboard si el registro fue exitoso."

¿Modelo, Vista o Controlador?

 **Respuestas:** A → **Modelo** (lógica de negocio) · B → **Vista** (presentación) · C → **Controlador** (flujo y coordinación)

UML Aplicado al Patrón MVC

El **Lenguaje Unificado de Modelado (UML)** es el estándar internacional para visualizar, especificar y documentar la arquitectura de sistemas de software. Aplicado a MVC, nos permite comunicar de forma precisa y profesional cómo se estructuran y comunican los componentes de nuestra aplicación.



Diagrama de Clases

Modela la estructura estática: clases, atributos, métodos y relaciones entre los componentes MVC. Fundamental para el diseño orientado a objetos.



Diagrama de Secuencia

Modela el comportamiento dinámico: el orden temporal de los mensajes entre objetos durante un caso de uso específico del sistema.



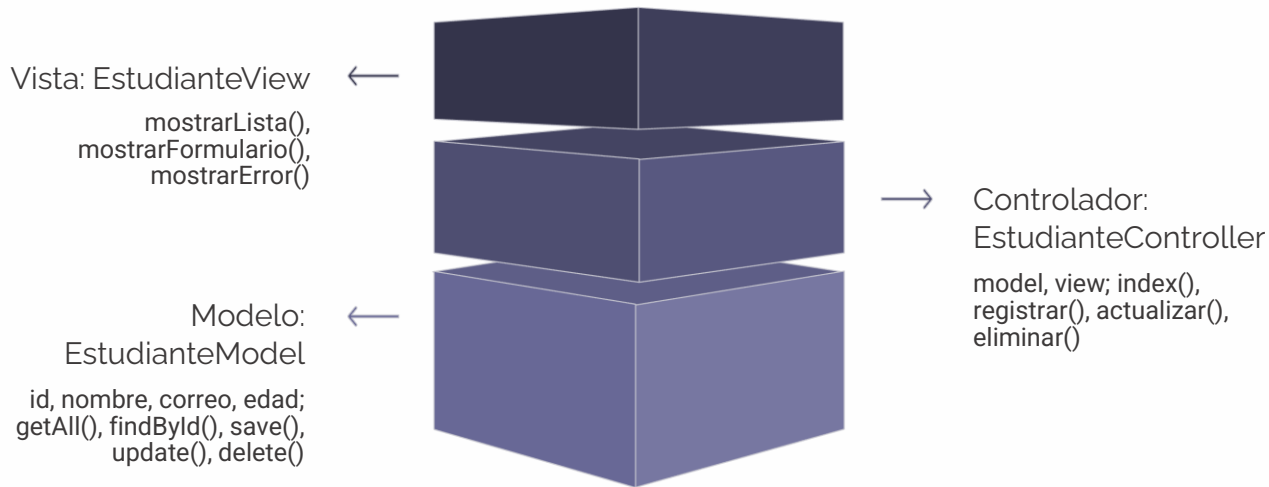
Diagrama de Componentes

Modela la arquitectura física: módulos de software, interfaces expuestas y las dependencias entre los grandes bloques del sistema.

Diagrama UML de Clases en MVC

El diagrama de clases muestra las **tres clases principales** de la arquitectura MVC y sus relaciones. La regla de navegabilidad establece que la dependencia fluye en una sola dirección:

Vista → **Controlador** → **Modelo**. Esto garantiza que el Modelo sea independiente y reutilizable.



Regla de Navegabilidad en Capas

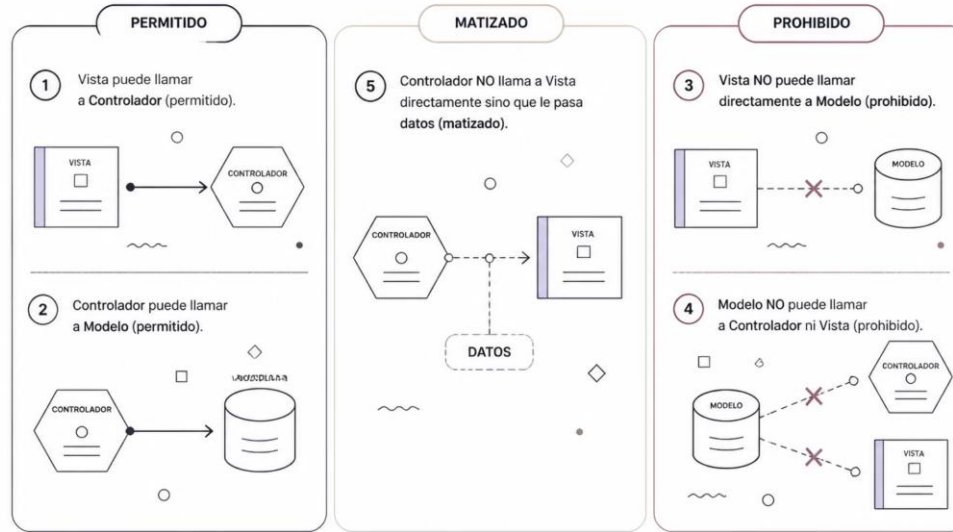
La flecha de dependencia apunta hacia la capa que se usa: **View** → **Controller** → **Model**. El Modelo no conoce al Controlador, y el Controlador no conoce la Vista directamente. Esta unidireccionalidad reduce el acoplamiento.

Tipos de Relaciones Comunes

Asociación: Controller usa Model · **Dependencia:** View depende de datos del Controller
· **Composición:** Controller contiene instancias de Model y View.

Regla de Navegabilidad: View → Controller → Model

La navegabilidad en capas es una de las reglas más importantes del patrón MVC. Determina qué clase puede conocer y llamar a otra. Violar esta regla genera acoplamiento indebido y rompe los beneficios del patrón.



Relación	¿Permitida?	Justificación
Vista → Controlador	✓ Sí	La Vista envía acciones al Controlador para ser procesadas
Controlador → Modelo	✓ Sí	El Controlador invoca la lógica de negocio del Modelo
Vista → Modelo	✗ No	Viola la separación de capas; crea acoplamiento directo
Modelo → Controlador	✗ No	El Modelo es independiente; no debe conocer el flujo

Diagrama UML de Secuencias en MVC

El diagrama de secuencias modela el **comportamiento dinámico** del sistema, mostrando el orden temporal exacto de los mensajes que se intercambian entre objetos para completar un caso de uso. Es ideal para documentar flujos como el inicio de sesión, el registro de datos o la búsqueda de información.

Elementos del Diagrama de Secuencia

Participantes (Lifelines)

Las entidades que intervienen en el flujo: Usuario, Vista, Controlador, Modelo, Base de Datos.

Mensajes

Llamadas sincrónicas ($\rightarrow$), asincrónicas ($\dashrightarrow$) y de retorno ($\dashleftarrow$ punteado) entre participantes.

Fragmentos combinados

Bloques condicionales (alt), bucles (loop) y referencias (ref) para flujos complejos.

¿Cuándo usar diagramas de secuencia?

- Documentar casos de uso complejos
- Comunicar flujos al equipo de desarrollo
- Identificar cuellos de botella en el diseño
- Revisar integración entre servicios o microservicios
- Detectar responsabilidades mal asignadas



En proyectos empresariales, el diagrama de secuencia es el primer artefacto que se revisa en una auditoría de arquitectura.

Secuencia: "Usuario Inicia Sesión en el Sistema"

Este es el flujo de secuencia completo para el caso de uso **"Inicio de Sesión"**, uno de los más frecuentes en cualquier sistema de información. Cada flecha representa un mensaje entre participantes en orden cronológico.

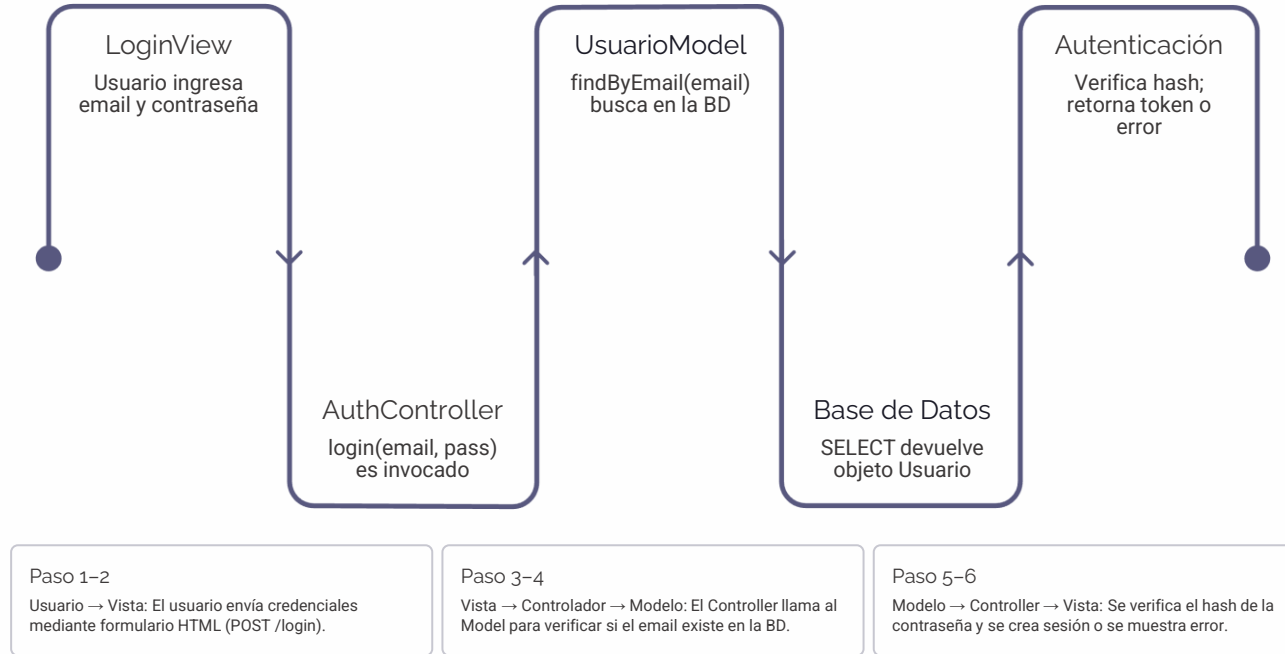
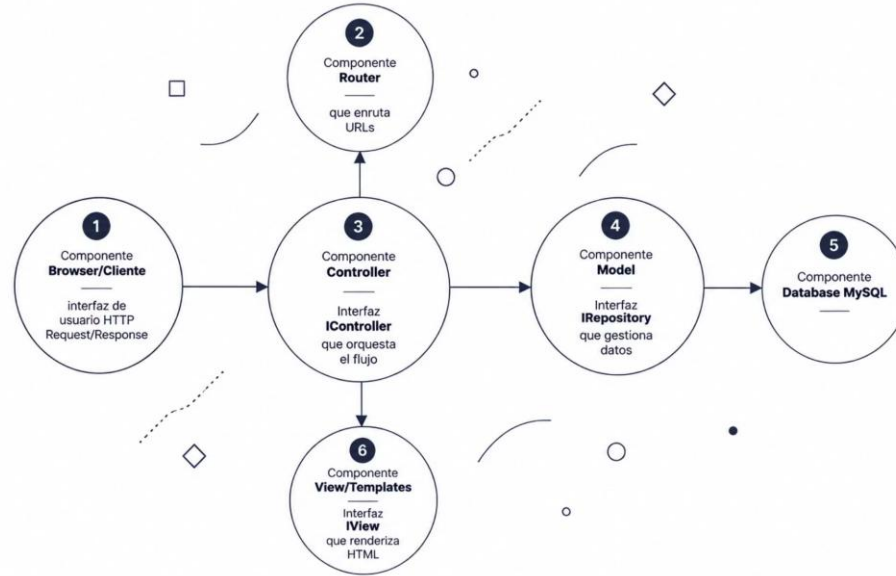


Diagrama UML de Componentes MVC

El diagrama de componentes modela la **arquitectura física del sistema**, mostrando los módulos de software, sus interfaces expuestas y las dependencias entre ellos. En MVC, cada capa es un componente con interfaces bien definidas que permite la sustitución o extensión independiente.



Componentes clave

- **Router**: Mapea URLs a Controladores
- **Controller**: Implementa la interfaz IController
- **Model**: Implementa IRepository (patrón)
- **View**: Motor de plantillas (Blade, Twig)
- **Database**: Capa de persistencia (MySQL, PostgreSQL)

Beneficio de las interfaces

Al definir interfaces (IRepository, IView), puedes cambiar la implementación concreta sin afectar los demás componentes. Por ejemplo: cambiar MySQL por MongoDB sin tocar el Controlador.

Sistema de Gestión de Estudiantes con MVC

Analizaremos cómo implementar las operaciones CRUD de un sistema de gestión académica siguiendo estrictamente el patrón MVC. Este es el tipo de sistema más frecuente en proyectos de carrera y en sistemas empresariales reales.



Registrar Estudiante

Vista: Formulario HTML con campos nombre, correo, matrícula.

Controller: valida datos y llama a `save()`.

Modelo: ejecuta `INSERT INTO estudiantes` y confirma.



Consultar Información

Vista: Tabla HTML paginada con resultados.

Controller: recibe parámetro de búsqueda y llama a `findBy()`.

Modelo: ejecuta `SELECT` con filtros y retorna colección.



Actualizar Datos

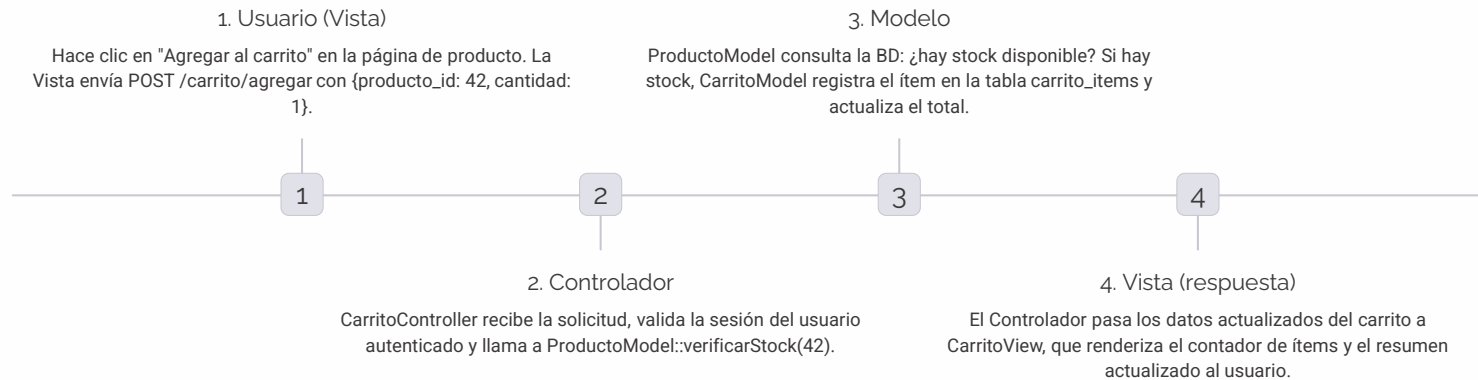
Vista: Formulario prellenado con datos actuales del estudiante.

Controller: recibe cambios y llama a `update(id, data)`.

Modelo: ejecuta `UPDATE estudiantes SET... WHERE id=?`.

Sistema de Tienda en Línea con MVC

Analizaremos el flujo completo cuando un usuario agrega un producto al carrito de compras, uno de los casos de uso más complejos en sistemas de comercio electrónico. Este ejemplo involucra gestión de estado de sesión, consultas de inventario y actualización de la vista en tiempo real.



```
// CarritoController.php
public function agregar(Request $req) {
    $producto = $this->productoModel->findById($req->producto_id);
    if ($producto->stock < $req->cantidad)
        return $this->view->render('error', ['msg' => 'Sin stock']);
    $this->carritoModel->agregar(session()->id(), $producto, $req->cantidad);
    return $this->view->render('carrito', $this->carritoModel->getResumen());
}
```

Buenas Prácticas y Conclusiones

El dominio del patrón MVC y el modelado UML son competencias fundamentales para cualquier ingeniero de sistemas. Estas buenas prácticas consolidan lo aprendido hoy y te preparan para proyectos reales de desarrollo de software profesional.

✓ Mantén las capas delgadas

Los Controladores deben ser delgados (thin controllers): no más de 10-15 líneas por método. Delega la lógica compleja siempre al Modelo o a clases de servicio.

✓ Un Modelo = Una responsabilidad

Cada clase Modelo debe representar una entidad de negocio. Evita los "GodModels" que manejan demasiadas entidades o responsabilidades no relacionadas.

✓ Documenta antes de programar

Los diagramas UML (clases, secuencias, componentes) deben crearse antes de escribir código. Detectar errores en papel es 10 veces más barato que en producción.

✓ Usa frameworks establecidos

Frameworks como Laravel, Spring Boot o Django implementan MVC de forma robusta y probada. Aprende sus convenciones en lugar de reinventar la rueda.

"La arquitectura de software bien diseñada no limita la creatividad del programador; la amplifica al eliminar la fricción innecesaria." — **Robert C. Martin**